18. Outbox Pattern

Tuesday, 28 April 2026 12:38 PM

Print

Before we go in deep with this EDA pattern: Lets first understand the PROBLEM

```
@PostMapping("/api/orders")
public ResponseEntity<String> createOrder(@RequestBody Order order)
{

    //Step1: Save to Database
    orderRepository.save(order);

    //Step2: Publish event to Kafka
    kafkaTemplate.send("order-events", order.getOrderId(), order);

    return ResponseEntity.ok("Order created");
}
```

Above code has one MAJOR FLAW:

We are writing to 2 separate independent systems:

a. DB
b. Kafka

There is no Transactions that spans both.

Failure Scenarios:

1. DB succeeds, Kafka Fails

```
orderRepository.save(order);  //Saved to DB
kafkaTemplate.send("order-events");  //Kafka is down
```

Result:

- Order exists in DB, but NO event published.
- Payment service never gets notified
- Inventory never decremented

This is DATA INCONSISTENCY.

2. Kafka succeeds, DB Fails

```
kafkaTemplate.send("order-events");  //Kafka event published
orderRepository.save(order);  //DB connection lost
```

Result:

- Event published but order doesn't exist in DB
- Payment service processes a non existent order.
- Inventory decremented for order which does not exist in DB.

This is DATA INCONSISTENCY and more impact than Use-case#1

This problem is known as DUAL WRITE problem

and during interviews, solutions generally provided is:

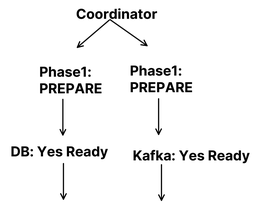
@Transactional + Manual Exception Throw

```
@Transactional
public void createOrder(OrderRequest request) {

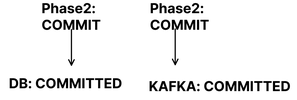
    // Step 1: DB write
    orderRepository.save(order);

     try {
      // Step 2: Kafka send but in try catch block, if kafka fails, DB is also Rollback
      kafkaTemplate.send("order-events", event).get();
    }
```

2PC (Two Phase Commit)



```
  catch (Exception e) {
    throw new RuntimeException("Kafka failed, rolling back!");  // rolls back DB
  }
}
```



This might looks correct at first glance, But loop hole is:

- What if Kafka Publish is success but we think it failed.

Timeline:
10:00 DB write -> success
10:05 kafkaTemplate.send() -> message sent to Kafka broker
10:06 Kafka broker receives message -> WRITES to partition
10:07 Kafka broker sends ACK back to your app
10:08 NETWORK ISSUE -> ACK packet lost in transit
10:09 Our app: "I never got ACK -> must have failed"
10:10 throw RuntimeException -> @Transactional rolls back DB

Result:
DB: Order DOES NOT exist (rolled back)
Kafka: Order event EXISTS

This might looks like a solution:

But, 2PC to work all participants need to understand the protocol. And KAFKA does not support 2PC.

So this solution pro and cons does not even matter as Kafka can not participate in 2PC, as it do not understand PREPARE, COMMIT commands from coordinator.

Solution is: OUTBOX PATTERN

Idea is:

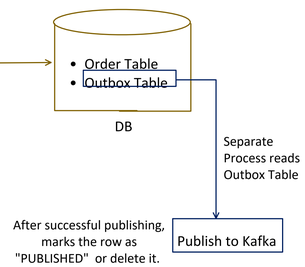
- Don't write directly to Kafka.
- Write to DB first and then a separate process read the DB and publish to kafka.

```
@Transactional
public void createOrder(OrderRequest request) {

    // Step 1: Order DB table write
    orderRepository.save(order);

    // Step 2: Event details stored in DB table first, known as Outbox Table
    outboxTableRepository.save(eventDetails);

}
```



Outbox Table Schema:

- Its just a regular DB table in our application.
- Contains columns which are required to successfully publish an event.

Most Common Schema Sample:

Id	Aggregate_type	Event_type	Aggregate_id	payload	Created_At	Is_published
101	Order	ORDER_CREATED	ORD-1	{...}		false

Id: Unique Id for this event. As producer can publish multiple time same event. So this can be used for Idempotency.

Aggregate_type: What entity this event is about like "ORDER" or "PAYMENT" etc. Used to determine which Kafka topic publish to.

Event_type: Tells what happened with entity like ORDER_CREATED, ORDER_CANCELLED etc.

Aggregate_id: This is used as message Key. As all related events should goes to the same partition, so this played a major role in maintaining order.

Payload: the actual data.

Created_At: when the event was created.

Optional field:

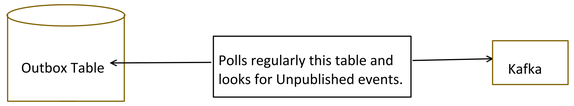
Is_published: False -> not yet sent to Kafka.
True -> sent to kafka. Can be cleaned later.

Print

Outbox Table is consumed in 2 ways:

1. **Polling: Is_published column is required.**
2. **CDC (Change Data Capture): Is_published column is not required.**

Polling:



OrderController.java

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {

@Autowired
private OrderService orderService;

@PostMapping
public String createOrder(@RequestBody Order order) {

        orderService.createOrder(order);
        return "Order created";
    }
}
```

OrderService.java

```
@Service
public class OrderService {

@Autowired
private OrderRepository orderRepository;

@Autowired
private OutboxRepository outboxRepository;

@Transactional
public Order createOrder(Order order) {

    // Step 1: save to orders table
    Order savedOrder = orderRepository.save(order);

    // Step 2: Create outbox event IN THE SAME TRANSACTION
    OutboxEvent outboxEvent = new OutboxEvent();
    outboxEvent.setAggregateType("Order");
    outboxEvent.setAggregateId(savedOrder.getOrderId());
    outboxEvent.setEventType("ORDER_CREATED");
    // Payload: JSON string of the event data
    String payload = String.format(
            "{\"orderId\":\"%s\"," +
            "\"quantity\":%d," +
            "\"totalAmount\":%.2f}",
            savedOrder.getOrderId(),
            savedOrder.getQuantity(),
            savedOrder.getTotalAmount()
        );
    outboxEvent.setPayload(payload);
    outboxRepository.save(outboxEvent);
    return savedOrder;
  }
}
```

No Kafka publish here, only DB tables touched, which can be part of 1 transaction.

OrderRepository.java

```
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {
}
```

OutboxRepository.java

```
@Repository
public interface OutboxRepository extends JpaRepository<OutboxEvent, String> {

@Query(value = """
SELECT * FROM outbox_events
WHERE published = false
ORDER BY created_at ASC
LIMIT :batchSize
""", nativeQuery = true)
List<OutboxEvent> findUnpublishedEvents(int batchSize);
}
```

Order.java (Entity)

```
@Entity
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "order_id")
    private String orderId;

    @Column
    private Integer quantity;

    @Column(name = "total_amount")
    private Double totalAmount;

    // Getters and Setters
}
```

application.properties

```
server.port=8085
spring.application.name=outbox-polling-demo

# PostgreSQL Connection
spring.datasource.url=jdbc:postgresql://localhost:5432/orderdb
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

# Kafka Producer Config
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.acks=all

# Outbox Poller Config (custom property)
outbox.polling.interval-ms=2000
outbox.polling.batch-size=10
```

OutboxEvent.java (Entity)

```
@Entity
@Table(name = "outbox_events")
public class OutboxEvent {

        @Id
        @Column
        private String id;

        @Column(name = "aggregate_type", nullable = false)
        private String aggregateType;

        @Column(name = "aggregate_id", nullable = false)
        private String aggregateId;

        @Column(name = "event_type", nullable = false)
        private String eventType;

        @Column(nullable = false, columnDefinition = "TEXT")
        private String payload;

        @Column(nullable = false)
        private boolean published;

        @Column(name = "created_at", nullable = false)
        private Instant createdAt;

        @Column(name = "published_at")
        private Instant publishedAt;

        public OutboxEvent() {
            this.id = UUID.randomUUID().toString();
            this.published = false;
            this.createdAt = Instant.now();
        }

        // --- Getters and Setters ---
}
```

OutboxPoller.java (the poller)

This scheduler is just 1 example, of poller.
We can have separate application itself which keep polling outbox table.
And once found the event can simply invoke the Publisher method.

```
@Component
public class OutboxPoller {

    @Autowired
    private OutboxRepository outboxRepository;
```

```
    @Autowired
    private KafkaTemplate<String, String> kafkaTemplate;

    @Value("${outbox.polling.batch-size}")
    private int batchSize;

    @Scheduled(fixedRateString = "${outbox.polling.interval-ms}")
    @Transactional
    public void pollAndPublish() {

        // Step 1: Fetch unpublished events
        List<OutboxEvent> events = outboxRepository.findUnpublishedEvents(batchSize);
        if (events.isEmpty()) {
            return;
        }

        for (OutboxEvent event : events) {
            try {
                String topic = event.getAggregateType().toLowerCase() + "-events";
                kafkaTemplate.send(topic, event.getAggregateId(), event.getPayload())
                             .get();
                event.setPublished(true);
                event.setPublishedAt(Instant.now());
                outboxRepository.save(event);
            }
            catch (Exception e) {
                    // BREAK to preserve ordering!
                    // If event 1 fails, we must NOT skip to event 2
                    break;
            }
        }
    }
}
```

This poller job runs every 2 seconds
Interval time configured in properties file.

Fetch events from outbox table.
In 1 go, it will fetch 10 records.
Batch size configured in properties file.

Publish to kafka

Update the published status to TRUE

This break is important, as event1 is failed, we should not continue to publish event2 , event3 and so on

As it will break ordering.

Disadvantages of Polling:

1. Multi-Instance handling:

When we have multiple instance, means multiple POLLERS, which could lead to:

- Duplicate processing of Rows:
  So need locking mechanism like *"SELECT ... FOR UPDATE SKIP LOCKED"*
- Ordering Risk:
  what if 2 POLLERS pick different rows which could lead to break ordering.
- ShedLock:
  To preserve ordering in multiple instance scenario, we need something like ShedLock technique.

2. Polling Gap:

   Event created at 10:00:00 but Poll runs at 10:00:02
   -> 2 sec delay

3. Constant DB load:
   Even when there is no events, Poller has to continuously query DB.
   This cause an extra CPU+IO overhead.

4. DB clean up:
   Outbox Event table need to be clean up regularly, as its size will grow rapidly.

5. Duplicate processing:
   Event published to kafka, but processed=True update failed.
   So during next run, again same event might get published, so Consumer must be IDEMPOTENT.

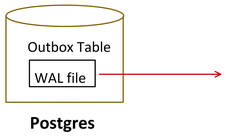
Another most scalable solution is: CDC (Change Data Capture)

CDC (Change Data Capture)

↓

Debezium (Tool which implements CDC)

CDC: It says don't ask the Database for changes, database will tell you.



WAL (Write-Ahead Logging):
Before INSERT/UPDATE/DELETE first WAL file is written and flushed to disk.
Then Table get updated.
This help Postgres in case recovery.

Also the same WAL file is used for Replication to followers.

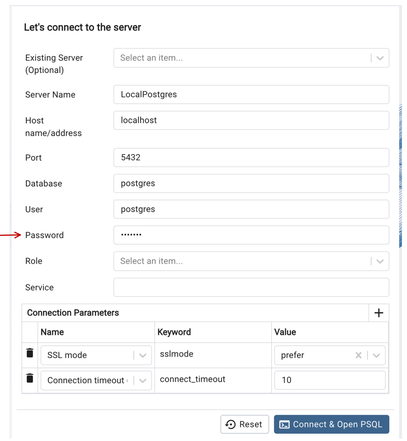
Similarly,
In MySQL its Binlog file which is use for replication to followers

CDC reads get these log files only from DB.

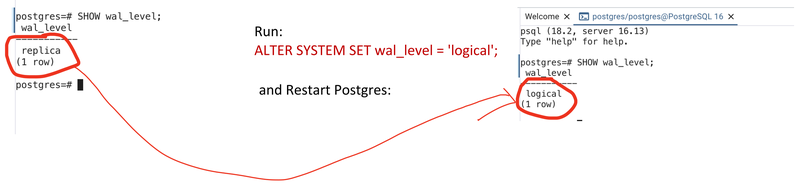
Step1: Download Postgres and start the server

https://www.postgresql.org/download/

pgAdmin (UI for postgres, automatically comes when we download Postgres)



While downloading, it ask us to provide the password, same need to be used here



Run:
ALTER SYSTEM SET wal_level = 'logical';

and Restart Postgres:

Need to change from Replica to Logical for CDC to work.

Replica: in WAL data is stored in Binary form

page5, offset 120 -> changed from X to Y

But this is of no use for CDC.

CDC need more info like which table, column etc.

Logical: in WAL data is still in Binary form + some extra metadata for decoding.

Table = order
Row id = 1
Column "Amount" changed from 100 to 200

Now CDC can publish event properly.

Print

Step2: Create our DB

```
CREATE DATABASE outbox_cdc;
```

Step3: Create our tables:

```
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  order_id VARCHAR(255) UNIQUE NOT NULL,
  quantity INT NOT NULL,
  total_amount DOUBLE PRECISION NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

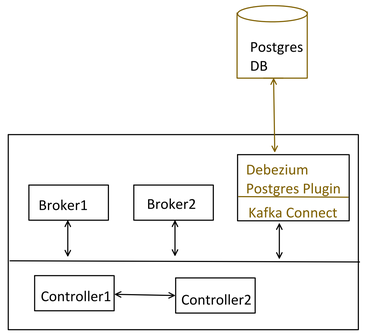
CREATE TABLE outbox_events (
  id VARCHAR(36) PRIMARY KEY,
  aggregate_type VARCHAR(255) NOT NULL,
  aggregate_id VARCHAR(255) NOT NULL,
  event_type VARCHAR(255) NOT NULL,
  payload TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Step4: Create Publication:

```
CREATE PUBLICATION cdc_publication FOR TABLE public.outbox_events;
```

Now PostgreSQL knows:
INSERT into orders -> NOT in publication -> skip
INSERT into outbox_events -> IN publication -> stream it
UPDATE users -> NOT in publication -> skip

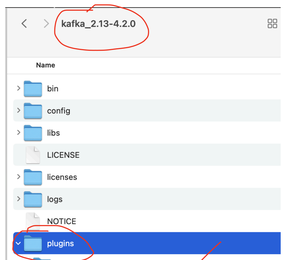
Step5: Now Download CDC tool (Debezium Postgres) plugin and add it in Kafka Connect



Download Debezium Postgres plugin:
https://repo1.maven.org/maven2/io/debezium/debezium-connector-postgres/3.5.0.Final/

Download this file:
debezium-connector-postgres-3.5.0.Final-plugin.tar.gz

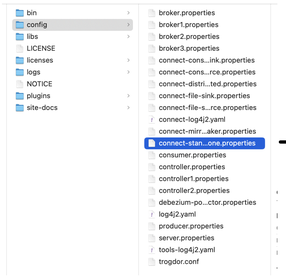
Now extract this Debezium plugin in "plugins" folder inside our kafka directory:





Print

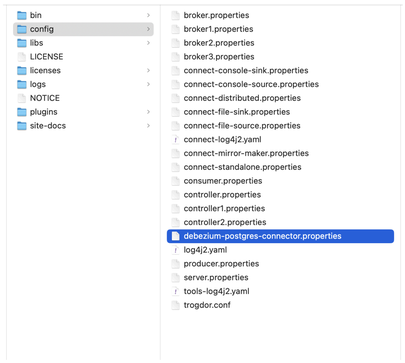
Now update the Kafka Connect configuration "*connect-standalone.properties*":



Add this:
plugin.path=/Users/shrayanshjain/Downloads/kafka_2.13-4.2.0/plugins

Now the last part, which connect both Debezium plugin and DB:

"debezium-postgres-connector.properties"



Create file: kafka/config/debezium-postgres-connector.properties

```
# unique name for this connector
name=outbox-connector

# Which Debezium plugin to use (Postgres, MySql etc.)
connector.class=io.debezium.connector.postgresql.PostgresConnector

# Database connection
database.hostname=localhost
database.port=5432
database.user=postgres
database.password=cdctest
database.dbname=outbox_cdc

# {prefix}.{schema}.{table}
topic.prefix=cdc

# Only watch the outbox_events table (ignore orders table)
table.include.list=public.outbox_events

# Replication slot name (Postgres will create this)
slot.name=debezium_slot

# Postgres used this field and it decodes the WAL before it send to Debezium
plugin.name=pgoutput

# Start with a snapshot of existing data, then stream
snapshot.mode=initial

publication.autocreate.mode=disabled
publication.name=cdc_publication

#any name we can give to transform : event router
transforms=outbox
 #debezium in build class for event routing
transforms.outbox.type=io.debezium.transforms.outbox.EventRouter

 #unique field in outbox_events table, so that will be used as Kafka message id
transforms.outbox.table.field.event.id=id
```

Print

```
#which column to use for key
transforms.outbox.table.field.event.key=aggregate_id

#which column to use for payload
transforms.outbox.table.field.event.payload=payload

#read column aggregate_type
transforms.outbox.route.by.field=aggregate_type
#in topic, we can add suffix
transforms.outbox.route.topic.replacement=${routedByValue}-events
```

When we start the Kafka Connect server -> Debezium registers itself as a follower at runtime, using the replication protocol.

Start:

- Controller
- Broker
- Kafka Connect :

bin/connect-standalone.sh config/connect-standalone.properties config/debezium-postgres-connector.properties

DEMO:

Query | Query History

```
INSERT INTO outbox_events (id, aggregate_type, aggregate_id, event_type, payload)
VALUES ('evt-005', 'Order', 'ORD-055', 'ORDER_CREATED',
        '{"orderId":"ORD-055","customerId":"CUST-001","amount":299.99}');
```

PUBLISHED TO KAFKA in topic "order-events"

```
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 % bin/kafka-dump-log.sh --deep-iteration --print-data-log \
  --files /tmp/broker1-logs/order-events-0/00000000000000000000.log
Dumping /tmp/broker1-logs/order-events-0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: -1 lastSequence: -1 producerId: -1 producerEpoch: -1 partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: Optio
nalLong.empty position: 0 CreateTime: 1777473454277 size: 161 magic: 2 compresscodec: none crc: 2137525922 isvalid: true
| offset: 0 CreateTime: 1777473454277 keySize: 9 valueSize: 73 sequence: -1 headerKeys: [id] key: "ORD-055" payload: "{\"orderId\":\"ORD-055\",\"customerId\":\"CUST-001\",\"amount\":299.99}"
shrayanshjain@Shrayanshs-MacBook-Pro-2 kafka_2.13-4.2.0 %
```